

Jour 1 — Matin : Chapitre 1 — Fondamentaux de l'IA et économie des LLM

Formation : IA pour développeur (2 jours) — Niveau : DÉBUTANT Durée : ~3h30
(matinée du jour 1) Pré-requis souhaités : avoir déjà ouvert un terminal et écrit quelques lignes de Python

À qui s'adresse ce chapitre ?

Vous êtes développeur (junior, confirmé, ou en reconversion) et vous **entendez parler d'IA partout** : ChatGPT, Copilot, Claude, Mistral, agents, RAG, MCP... Vous voulez **comprendre ce qui se cache derrière** et **commencer à l'utiliser concrètement** dans votre travail.

Bonne nouvelle : vous n'avez **pas besoin** d'être mathématicien ni datascientist. Vous avez juste besoin :

- de savoir lire du Python simple (boucles, fonctions),
- d'être à l'aise avec un terminal,
- d'avoir envie de **manipuler** plutôt que d'écouter passivement.

Objectifs pédagogiques (ce que vous saurez faire à la fin)

À la fin de ce chapitre, vous serez capable de :

1. Expliquer **avec vos mots** ce qu'est un LLM (à un collègue non-tech).
2. Décrire les briques de base : neurone, poids, biais, apprentissage.
3. Faire la différence entre **un gros modèle, un petit modèle, et un modèle distillé**.
4. **Calculer combien coûte** un appel à une IA (input, output, cache).
5. Comprendre pourquoi un message trop long peut coûter cher ou être moins bien compris.
6. Installer un **assistant IA dans votre IDE** (VS Code ou autre).
7. Choisir entre **API en ligne** et **modèle local** selon votre situation.

Plan de la matinée

Heure	Section	Durée
09:00	1. Glossaire & analogie « la chaîne de cuisine »	20 min

Heure	Section	Durée
09:20	2. Du neurone au LLM (très progressif)	30 min
09:50	3. Comment une IA apprend ?	25 min
10:15	Pause	15 min
10:30	4. Petit, gros, distillé : choisir un modèle	20 min
10:50	5. Les tokens, la facture, le cache	30 min
11:20	6. Limites : context window et rate limits	15 min
11:35	TP 1.1 — Installer un assistant IA	25 min
12:00	TP 1.2 — Calculer une facture LLM	30 min
12:30	Pause déjeuner	—

1. Glossaire de démarrage

Avant de plonger, **familiarisez-vous avec ces 10 mots**. On va les utiliser tout au long de la formation.

Mot	Définition courte (à expliquer à votre grand-mère)
IA générative	Une IA qui produit du nouveau contenu (texte, image, code) plutôt que de classer ou prédire un chiffre.
LLM (<i>Large Language Model</i>)	Une « grosse IA qui parle » — entraînée sur des milliards de pages de texte.
SLM (<i>Small Language Model</i>)	Une « petite IA qui parle » — moins puissante mais qui tourne sur votre laptop.
Prompt	Le message que vous envoyez à l'IA. Comme une consigne donnée à un freelance.
Token	Un morceau de texte (≈ 3-4 caractères). L'IA compte en tokens , pas en mots.
Context window	La « mémoire courte » de l'IA — combien de tokens elle peut lire en une fois.
API	Un point d'entrée web qui permet à votre code d'appeler une IA distante.
Modèle	Un fichier (gigantesque, plusieurs Go) qui contient tout ce que l'IA a appris.
Inférence	L'action de faire répondre un modèle à une question (par opposition à l'entraînement).
Hallucination	Quand l'IA invente une information avec aplomb. C'est le risque numéro 1 .

💡 **Astuce** : revenez à ce glossaire dès qu'un mot vous échappe. Le vocabulaire est 50 % de la difficulté en IA.

2. Du neurone au LLM — analogie de la chaîne de cuisine

Analogie : un restaurant à plusieurs cuistots

Imaginez **un restaurant** où plusieurs cuisiniers travaillent à la chaîne :

- Le **commis** lave les légumes et donne le résultat au cuistot suivant.
- Le **second** coupe les légumes lavés et passe au cuistot suivant.
- Le **chef de partie** assaisonne.
- Le **chef** dresse l'assiette finale.

Chaque cuistot **reçoit quelque chose, le transforme, et passe au suivant**. Si chacun fait bien son travail, l'assiette finale est bonne.

Un réseau de neurones, c'est pareil : chaque « neurone » reçoit des chiffres, les transforme un peu, et passe au suivant. À la fin, vous obtenez une réponse.

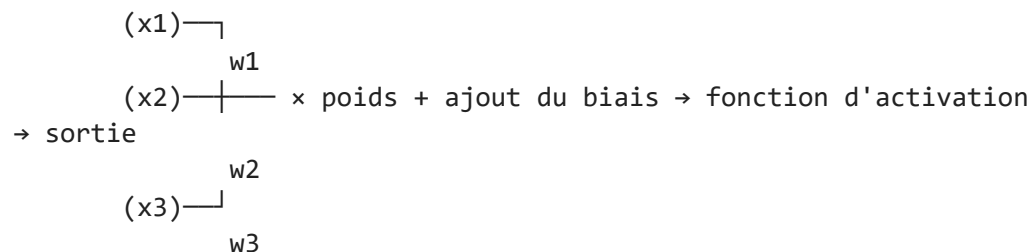
Et un LLM ?

Un LLM est **un très grand restaurant** : des centaines de couches de cuistots, des **milliards de paramètres** (= recettes que chaque cuistot a apprises), spécialisé dans **la cuisine du texte**.

Quand vous écrivez « *Bonjour, comment ça...* », le LLM est très fort pour prédire que le mot suivant le plus probable est « *va* ». C'est aussi simple — et aussi puissant — que ça : **un LLM prédit le prochain morceau de texte**.

Le « neurone » mathématique (très simplifié)

Si on ouvre le capot d'un neurone, voici ce qu'il fait :



En 3 étapes :

1. **Recevoir** plusieurs nombres d'entrée (x1, x2, x3...).
2. **Multiplier** chacun par un **poids** appris (w1, w2, w3...) et **additionner**.
3. Ajouter un **biais** **b** (un petit décalage).
4. Passer le résultat dans une **fonction** (sigmoid, ReLU...) qui « tasse » la valeur.

Voilà, c'est tout. Un neurone, c'est juste **sortie = f(somme des entrées × poids + biais)**.

🧠 **À retenir** : les **poids** et les **biais** sont des chiffres. Un LLM, ce sont **des milliards de ces chiffres** qui ont été ajustés pendant l'entraînement. GPT-4, c'est $\approx 1\,000$ milliards de chiffres.

```
In [ ]: # =====
# Démo 1 – Un neurone unique, codé à la main
# Objectif : voir qu'un neurone n'est qu'une formule mathématique
# =====

# On importe numpy, une librairie qui gère les tableaux de chiffres
import numpy as np

# --- Étape 1 : on définit la fonction d'activation "sigmoid"
# La sigmoid transforme n'importe quel nombre en un nombre entre 0 et 1
# Utile pour répondre "oui" (proche de 1) ou "non" (proche de 0)
def sigmoid(z):
    return 1 / (1 + np.exp(-z))    # formule classique e^-z

# --- Étape 2 : on définit notre neurone
def neurone(entrees, poids, biais):
    # entrees x poids = produit terme à terme, puis on additionne tout : c'est le
    somme = np.dot(entrees, poids)
    # On ajoute le biais (un décalage)
    somme_decalee = somme + biais
    # On passe dans la sigmoid pour obtenir une sortie entre 0 et 1
    return sigmoid(somme_decalee)

# --- Étape 3 : on essaie avec des valeurs concrètes
# Imaginez 3 entrées : taille (m), poids (kg), âge (années)
entrees = np.array([1.75, 70, 35])    # quelqu'un d'1m75, 70 kg, 35 ans
poids = np.array([0.1, 0.05, -0.02]) # poids "appris" par l'IA (ici on les i
biais = -2.5                          # biais "appris"

# --- Étape 4 : on calcule la sortie
sortie = neurone(entrees, poids, biais)
print(f"Sortie du neurone : {sortie:.3f}") # affichage à 3 décimales

# Interprétation : la sortie est proche de 1 → "oui" (par ex. "est-ce un adulte ?")
# Si la sortie est proche de 0 → "non"
# Les poids et le biais ont été choisis pour que la réponse soit cohérente
```

Pourquoi est-ce que ça « apprend » ?

Au début, les poids et biais sont **aléatoires** → l'IA répond n'importe quoi.

L'**entraînement** consiste à montrer à l'IA des millions d'exemples avec la **bonne réponse**, et à **ajuster** les poids petit à petit pour réduire les erreurs.

🎯 **Analogie** : c'est comme apprendre à viser au tir à l'arc.

- 1er tir : vous ratez la cible (poids aléatoires).
- Vous regardez **où est tombée la flèche** par rapport à la cible (= erreur).
- Vous ajustez votre posture (= ajustement des poids).
- Vous répétez 1 million de fois.
- À la fin, vous touchez la cible.

C'est ce qu'on appelle la **descente de gradient** : on regarde l'erreur, on calcule **dans quelle direction** ajuster les poids, et on le fait, encore et encore.

3. Comment une IA apprend — démo concrète

On va apprendre à un mini-modèle à deviner une formule très simple : $y = 3x + 2$.

L'IA ne **connaît pas** la formule. Elle doit la **deviner** à partir d'exemples.

```
In [ ]: # =====
# Démo 2 – Descente de gradient sur la formule  $y = 3x + 2$ 
# =====

import numpy as np

# --- Étape 1 : on crée des données d'entraînement
# rng = générateur de nombres aléatoires reproductible (seed=42)
rng = np.random.default_rng(42)

# On génère 100 valeurs de x aléatoires (chiffres autour de 0)
X = rng.normal(size=100)

# On calcule la vraie réponse  $y = 3x + 2$ , plus un peu de bruit (= imperfection)
y_reel = 3 * X + 2 + rng.normal(scale=0.2, size=100)

# --- Étape 2 : on initialise notre modèle avec des poids ALÉATOIRES
w = 0.0 # poids initial (devrait converger vers 3)
b = 0.0 # biais initial (devrait converger vers 2)

# --- Étape 3 : on choisit le "learning rate" = pas d'apprentissage
# Trop grand = on oscille. Trop petit = on apprend lentement.
lr = 0.05

# --- Étape 4 : on boucle, on regarde l'erreur, on ajuste
for epoch in range(200): # 200 itérations d'entraînement
    # 1. Prédiction actuelle du modèle
    y_pred = w * X + b

    # 2. Erreur = différence entre prédiction et vérité
```

```

erreur = y_pred - y_reel

# 3. Calcul du gradient (= direction d'ajustement)
# Formule classique pour une régression linéaire
grad_w = (2 / len(X)) * np.dot(erreur, X)
grad_b = (2 / len(X)) * erreur.sum()

# 4. On ajuste : on soustrait le gradient × Learning rate
w -= lr * grad_w
b -= lr * grad_b

# 5. Tous les 50 epochs, on affiche où on en est
if epoch % 50 == 0:
    print(f"Epoch {epoch:3d} | w = {w:.3f} (cible 3) | b = {b:.3f} (cible 2)")

print(f"\nFinal      | w = {w:.3f} (cible 3) | b = {b:.3f} (cible 2)")
# On voit que le modèle "retrouve" approximativement la formule de départ !

```

Ce qu'on vient de voir

Vous avez **vu un mini-modèle apprendre tout seul** une formule à partir de données. Un LLM, c'est **exactement le même principe**, mais :

- Des **milliards de paramètres** au lieu de 2 (`w` et `b`).
- Des **téraoctets de texte** au lieu de 100 chiffres.
- Des **GPU** pendant des mois au lieu de votre laptop pendant 1 seconde.
- Une architecture spéciale appelée **Transformer** au lieu d'une simple ligne.

Mais la **logique fondamentale est la même**. Vous venez de comprendre l'essentiel.

✅ **Checkpoint** : sauriez-vous expliquer en 3 phrases ce que fait la descente de gradient ? Si oui → continuez. Si non → relisez la cellule ci-dessus.

4. Petit, gros, distillé : quel modèle choisir ?

Les 3 grandes familles

Famille	Taille	Où ça tourne ?	Pour quoi faire ?
LLM (gros)	> 30 Mds paramètres	Cloud (API)	Tâches complexes, raisonnement long
SLM (petit)	< 10 Mds paramètres	Votre laptop ou serveur perso	Autocomplete, résumé, classification
Distillé	Variable, souvent petit	Cloud ou local	Quand on veut un petit modèle qui imite un gros

Analogie : la voiture

- **LLM gros** = Mercedes Classe S. Tout va bien, mais ça coûte cher et il faut faire le plein.
- **SLM petit** = Renault Clio. Suffit pour aller au boulot. Économique. Pas adapté aux longs voyages avec 7 personnes.
- **Modèle distillé** = Clio E-Tech (modèle compact qui essaie d'imiter la Classe S). Compromis.

Exemples 2026

Modèle	Famille	Mémoire requise
Claude Opus 4.6	LLM gros	API uniquement
GPT-4o	LLM gros	API uniquement
Llama-3.1-8B	SLM moyen	16 Go RAM
Phi-3-mini (3,8B)	SLM petit	8 Go RAM
Claude Haiku 4.5	Distillé (probablement)	API uniquement
Mistral-7B	SLM moyen	16 Go RAM

Quand choisir quoi ?

Vous voulez...	Choisissez
Coder un agent qui doit raisonner sur 50 fichiers	LLM gros (API)
Compléter du code dans votre IDE	SLM local (gratuit, rapide)
Résumer un document interne confidentiel	SLM local (vos données ne sortent pas)
Faire tourner une démo gratuite sur votre site	SLM local sur votre serveur
Avoir la meilleure qualité possible	LLM gros (API, plus cher)

💡 **Règle empirique** : commencez avec un **SLM local** pour apprendre, passez à l'**API** quand vous avez besoin de plus de puissance.

5. Les tokens : comprendre la facture

Qu'est-ce qu'un token ?

Un **token** est un **morceau de mot** ou un mot court. C'est l'unité que l'IA manipule.

Quelques règles d'estimation :

- **En anglais** : 1 token $\approx$ 4 caractères $\approx$ 0.75 mot.
- **En français** : 1 token $\approx$ 3 caractères $\approx$ 0.6 mot (le français consomme **plus** de tokens).
- **Code** : variable, mais souvent $\approx$ 1 token par identifiant ou symbole.

Exemples concrets

Texte	Tokens approximatifs
"Hello world"	2 tokens
"Bonjour le monde"	4-5 tokens
"def fibonacci(n):"	5 tokens
Un email de 200 mots en français	~ 300 tokens
Un fichier Python de 500 lignes	~ 5 000 à 8 000 tokens
Un PDF de 50 pages	~ 25 000 à 35 000 tokens

```
In [ ]: # =====
# Démo 3 – Compter des tokens dans une vraie phrase
# =====
# Pour compter exactement, on utilise la librairie "tiktoken"
# (utilisée par OpenAI, mais marche aussi comme estimation pour les autres)

# --- Étape 0 : installer si besoin (à décommenter au premier usage)
# !pip install tiktoken

# --- Étape 1 : importer la librairie
import tiktoken

# --- Étape 2 : choisir un encodeur (cl100k_base = celui de GPT-4 / Claude)
encodeur = tiktoken.get_encoding('cl100k_base')

# --- Étape 3 : compter des phrases différentes
phrases = [
    "Hello world",
    "Bonjour le monde",
    "Comment ça va aujourd'hui ?",
    "def add(a: int, b: int) -> int:\n    return a + b",
    "Je vais te raconter une longue histoire qui prendra plusieurs lignes...",
]

# On boucle sur chaque phrase et on compte ses tokens
for phrase in phrases:
    # encode() retourne la liste des IDs de tokens
    ids_tokens = encodeur.encode(phrase)
    nb_tokens = len(ids_tokens)
    # On affiche : phrase | nb tokens | nb caractères
    print(f"{nb_tokens:3d} tokens | {len(phrase):3d} caracteres | {phrase[:50]!r}")
```

Les 3 lignes de la facture

Quand vous appelez une API d'IA, **on vous facture sur 3 compteurs** :

Compteur	À quoi ça correspond	Prix relatif
Input (entrée)	Le prompt que vous envoyez	prix de base
Output (sortie)	Le texte que l'IA génère	5 à 15× plus cher que l'input
Cache	Une partie du prompt réutilisée entre appels	5 à 10× moins cher que l'input

Exemple chiffré (Claude Sonnet 4 en 2026, à titre indicatif)

- Input : **3 USD** par million de tokens
- Output : **15 USD** par million de tokens
- Cache (lecture) : **0,30 USD** par million de tokens

Le « cache » : la magie qui économise

Si vous avez toujours le même prompt système (par exemple « *Tu es un assistant Python qui répond en français...* » de 2 000 tokens), vous pouvez le **mettre en cache**.

Sans cache : vous payez 2 000 tokens d'input à chaque appel. **Avec cache** : vous payez 2 000 tokens d'input **la première fois**, puis 2 000 tokens de cache (10× moins cher) les fois suivantes.

Pour un volume de 10 000 appels/mois → **économie réelle de 50 à 200 USD/mois**.

```
In [ ]: # =====
# Démo 4 – Calculatrice de coût LLM
# =====

# --- Étape 1 : définir Les tarifs (en USD pour 1 MILLION de tokens)
PRIX_INPUT      = 3.0      # 3 USD / million de tokens d'entrée
PRIX_OUTPUT     = 15.0     # 15 USD / million de tokens de sortie (plus cher !)
PRIX_CACHE_READ = 0.30     # 0.30 USD / million de tokens lus depuis cache (10x moins cher)

# --- Étape 2 : fonction qui calcule Le coût d'UN appel
def cout_un_appel(nb_input, nb_output, nb_cache=0):
    """Calcule le coût en USD d'un seul appel LLM.

    Args:
        nb_input : nombre de tokens d'entrée TOTAL (incluant ceux du cache)
        nb_output : nombre de tokens de sortie
        nb_cache : nombre de tokens lus depuis le cache (forcément <= nb_input)

    Returns:
        Coût en USD pour cet appel.
    """
    # 1. Tokens d'input qui ne viennent PAS du cache (donc payés au tarif plein)
    input_paye_plein = nb_input - nb_cache

    # 2. Conversion en millions et multiplication par Le prix
    cout_input  = (input_paye_plein / 1_000_000) * PRIX_INPUT
    cout_output = (nb_output / 1_000_000) * PRIX_OUTPUT
```

```

cout_cache = (nb_cache / 1_000_000) * PRIX_CACHE_READ

# 3. On somme les 3
return cout_input + cout_output + cout_cache

# --- Étape 3 : on essaie sur un cas concret
# Imaginons un agent qui reçoit 3000 tokens d'input et répond avec 500 tokens
cout_sans_cache = cout_un_appel(nb_input=3000, nb_output=500)
print(f"Appel SANS cache : {cout_sans_cache:.5f} USD")

# Maintenant avec 2500 tokens venant du cache
cout_avec_cache = cout_un_appel(nb_input=3000, nb_output=500, nb_cache=2500)
print(f"Appel AVEC cache : {cout_avec_cache:.5f} USD")

# Différence en pourcentage
gain_pct = (cout_sans_cache - cout_avec_cache) / cout_sans_cache * 100
print(f"Economie grace au cache : {gain_pct:.1f} %")

# --- Étape 4 : on multiplie par un volume mensuel
nb_appels_mois = 10_000
print(f"\nSur {nb_appels_mois} appels/mois :")
print(f"  Sans cache : {cout_sans_cache * nb_appels_mois:.2f} USD/mois")
print(f"  Avec cache : {cout_avec_cache * nb_appels_mois:.2f} USD/mois")
print(f"  Economie : {(cout_sans_cache - cout_avec_cache) * nb_appels_mois:.2f} U

```

6. Context window et rate limits

Context window : la « mémoire courte » de l'IA

Chaque modèle a une **limite maximale** de tokens qu'il peut traiter en une fois (input + output).

Modèle	Context window
Claude Sonnet 4	200 000 tokens (~500 pages)
GPT-4o	128 000 tokens
Gemini 1.5 Pro	1 000 000 tokens
Mistral 7B (local)	32 000 tokens
Llama-3.1-8B (local)	128 000 tokens

⚠ **Attention** : un context **très long ne veut pas dire meilleure qualité**. Au contraire, les modèles ont tendance à « **oublier le milieu** » d'un très long contexte (effet *lost in the middle*). **Règle pratique** : envoyez **le minimum nécessaire**, pas tout ce que vous avez.

Rate limits : combien d'appels par minute ?

Les fournisseurs limitent :

- **RPM** (Requests Per Minute) : combien de requêtes par minute. Ex : 50 à 5 000.
- **TPM** (Tokens Per Minute) : combien de tokens par minute. Ex : 20 000 à 4 000 000.
- **TPD** (Tokens Per Day) : quota journalier.

Si vous dépassez → erreur **HTTP 429** (« Too Many Requests »). À gérer dans votre code avec un **back-off** (attendre puis réessayer).

```
In [ ]: # =====
# Démo 5 - Gérer un rate limit avec un back-off exponentiel
# =====

import time
import random # juste pour simuler des erreurs

# --- Fonction qui SIMULE un appel API qui peut échouer
def appel_api_simule(prompt):
    # On simule un échec 30 % du temps
    if random.random() < 0.3:
        raise Exception("HTTP 429: Too Many Requests")
    return f"Réponse à : {prompt[:30]}..."

# --- Fonction avec gestion d'erreur et back-off exponentiel
def appel_robuste(prompt, max_essais=5):
    """Essaie jusqu'à max_essais fois, en attendant de plus en plus."""
    # On boucle sur le nombre d'essais
    for essai in range(max_essais):
        try:
            # On tente l'appel
            return appel_api_simule(prompt)
        except Exception as e:
            # On affiche l'erreur
            print(f" Essai {essai+1} echoue : {e}")
            # Si ce n'est pas le dernier essai, on attend
            if essai < max_essais - 1:
                # Back-off EXPONENTIEL : 1s, 2s, 4s, 8s, 16s...
                attente = 2 ** essai
                print(f" Attente de {attente}s avant nouvelle tentative...")
                time.sleep(attente)
            # Si tous les essais ont échoué :
            raise Exception("Echec apres tous les essais")

# --- On essaie
random.seed(123) # pour reproductibilité de la démo
resultat = appel_robuste("Quel est le sens de la vie ?")
print(f"\nResultat final : {resultat}")
```

TP 1.1 — Installer votre premier assistant IA dans votre IDE

Durée : 25 minutes — Niveau : débutant

Pourquoi ce TP ?

Pour la suite de la formation, vous allez **utiliser un assistant IA dans votre IDE** (VS Code, JetBrains, Cursor...). Il y a 3 grandes options. On vous recommande l'**option A** : c'est **gratuit, local, et confidentiel** (vos données ne sortent pas de votre laptop).

Option A — Ollama + Continue (recommandé)

Pré-requis : RAM $\geq$ 16 Go (sinon prenez un modèle plus petit comme `phi3:mini`).

Étape 1 — Installer Ollama (gestionnaire de modèles locaux)

- macOS / Linux :
`curl -fsSL https://ollama.com/install.sh | sh`
- Windows : télécharger l'installateur sur <https://ollama.com>

Étape 2 — Télécharger un modèle (4,5 Go environ)

```
ollama pull llama3.1:8b
```

Si votre laptop est plus modeste :

```
ollama pull phi3:mini    # 2,3 Go, plus Léger
```

Étape 3 — Tester Ollama en ligne de commande

```
ollama run llama3.1:8b
```

```
>>> Bonjour, peux-tu te présenter en 3 phrases ?
```

Étape 4 — Installer Continue dans VS Code

- Aller dans l'onglet *Extensions* (Ctrl+Shift+X)
- Chercher *Continue* (auteur : Continue.dev)
- Cliquer *Install*

Étape 5 — Configurer Continue

- Ouvrir le fichier `~/.continue/config.json` (Cmd/Ctrl + Shift + P → *Continue: Open Config*)
- Remplacer par la config ci-dessous (voir cellule suivante)

Étape 6 — Tester dans un fichier Python

- Créer `tva.py`
- Taper en haut : `# fonction qui calcule la TVA à 20 %`
- Attendre 2 secondes → l'IA propose la complétion
- Appuyer sur **Tab** pour accepter

Option B — GitHub Copilot

Si vous avez un compte étudiant ou un abonnement → installer l'extension *GitHub Copilot* dans VS Code, se connecter, c'est tout.

Option C — Cursor

Télécharger Cursor (<https://cursor.com>), il intègre tout par défaut (essai gratuit 14 jours).

Livrables du TP

1. Une capture d'écran de votre IDE avec **une complétion acceptée**.
2. Un fichier `tva.py` généré par l'IA.
3. **3 lignes** dans `mes_notes.md` : *qu'est-ce que j'ai trouvé facile / difficile / surprenant ?*

```
In [ ]: # =====
# Fichier ~/.continue/config.json (à copier-coller dans votre éditeur)
# =====
# Cette cellule génère Le JSON. Copiez la sortie dans votre fichier.

import json

config = {
    # "models" = liste des modèles que vous voulez utiliser
    "models": [
        {
            "title": "Llama 3.1 8B local",      # nom affiché dans Continue
            "provider": "ollama",              # on parle à Ollama
            "model": "llama3.1:8b",            # Le modèle téléchargé
            "apiBase": "http://localhost:11434" # Ollama écoute sur ce port
        }
    ],
    # "tabAutocompleteModel" = modèle pour l'autocomplétion (Tab)
    "tabAutocompleteModel": {
        "title": "Llama autocomplete",
        "provider": "ollama",
        "model": "llama3.1:8b"
    }
}

# Affiche Le JSON joliment formaté (4 espaces d'indentation)
print(json.dumps(config, indent=4))
```

TP 1.2 — Calculer une facture LLM réelle

Durée : 30 minutes — Niveau : débutant

Scénario

Vous êtes lead dev. Votre équipe utilise **Claude Sonnet 4** pour générer automatiquement des tests unitaires à chaque PR.

Volume actuel :

- 1 200 PR/mois
- 8 000 tokens d'input en moyenne (code modifié + diff + consigne système)
- 2 500 tokens d'output en moyenne (les tests générés)

Questions à résoudre

Q1. Combien coûte le service actuellement par mois ?

Q2. Si on active le **prompt caching** (5 000 tokens d'input fixes = consigne système + style guide), combien on économise ?

Q3. Si on bascule sur **Claude Haiku 4.5** (input 0,80 USD/Mtok, output 4 USD/Mtok), combien on paye ? Quel risque ?

Q4. Si on **limite max_tokens à 1 500** au lieu de 2 500 (parfois la réponse est tronquée), combien on paye ?

Q5. Quelle est **votre recommandation** ? (1 paragraphe de 5 lignes)

Squelette de calcul à compléter

```
In [ ]: # =====
# TP 1.2 - A COMPLETER
# =====

# --- Données du scénario
PR_PAR_MOIS = 1200
TOKENS_IN = 8000      # tokens d'input par appel
TOKENS_OUT = 2500     # tokens d'output par appel

# --- Tarifs (USD par million de tokens)
# Claude Sonnet 4
SONNET_IN, SONNET_OUT, SONNET_CACHE = 3.0, 15.0, 0.30
# Claude Haiku 4.5
HAIKU_IN, HAIKU_OUT = 0.80, 4.0

# --- A vous : implémentez la fonction
def cout_appel(t_in, t_out, prix_in, prix_out, t_cache=0, prix_cache=0):
    """Calcule le coût d'UN appel en USD."""
    # TODO : voir l'exemple de La "Démonstration 4" plus haut
    pass

# --- Q1 : Sonnet sans cache
# c_q1 = cout_appel(...) * PR_PAR_MOIS
# print(f"Q1 : {c_q1:.2f} USD/mois")
```

```
# --- Q2 : Sonnet avec 5000 tokens en cache
# c_q2 = cout_appel(...) * PR_PAR_MOIS
# print(f"Q2 : {c_q2:.2f} USD/mois")

# --- Q3 : Haiku (sans cache)
# c_q3 = cout_appel(...) * PR_PAR_MOIS
# print(f"Q3 : {c_q3:.2f} USD/mois")

# --- Q4 : Sonnet avec max_tokens 1500
# c_q4 = cout_appel(...) * PR_PAR_MOIS
# print(f"Q4 : {c_q4:.2f} USD/mois")

# Corrigé disponible dans corriges/jour1/CORRIGE_chapitre1.ipynb
```

Synthèse de la matinée (à relire avant la pause déjeuner)

Concept	À retenir en une phrase
LLM	Un gros « prédicteur de prochain mot » entraîné sur des téraoctets de texte.
Neurone	<code>sortie = f(somme des entrées × poids + biais)</code> .
Apprentissage	On ajuste les poids pour réduire l'erreur, par descente de gradient.
Token	L'unité de comptage. ≈ 3-4 caractères.
3 compteurs de facturation	input (base) / output (5-15× plus cher) / cache (10× moins cher).
Context window	Limite de tokens par appel. Plus grand ≠ meilleur.
Rate limit	Limite d'appels/minute. À gérer avec back-off exponentiel.
Modèle local vs API	Local = gratuit, privé, moins puissant. API = puissant, payant, données qui sortent.

 Bon appétit ! On se retrouve à 14h pour la suite : **comment écrire de bons prompts pour le code.**